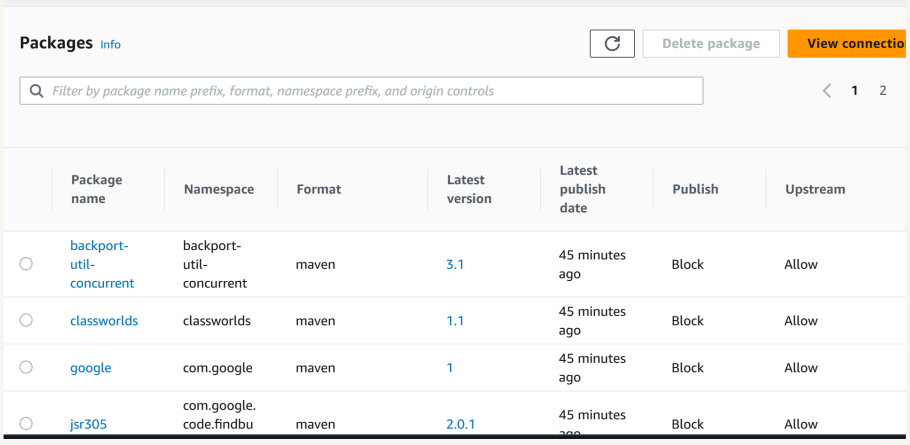
nextwork.org

Secure Packages with CodeArtifact

Kayira Bertrand
https://community.nextwork.org/u/ece9d43c



Packages Info

Delete package | View connectio

Filter by package name prefix, format, namespace prefix, and origin controls

1 2

Package name	Namespace	Format	Latest version	Latest publish date	Publish	Upstream
backport-util-concurrent	backport-util-concurrent	maven	3.1	45 minutes ago	Block	Allow
classworlds	classworlds	maven	1.1	45 minutes ago	Block	Allow
google	com.google	maven	1	45 minutes ago	Block	Allow
jsr305	com.google.code.findbu	maven	2.0.1	45 minutes	Block	Allow

Kayira Bertrand
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate how to set up CodeArtifact as our CI/CD Pipeline's Artifact repository! I'm doing this project to learn the importance and value of having an artifact repository. It is a huge time saver.

Key tools and concepts

Services I used were CodeArtifact, Amazon EC2, IAM, GitHub and VS Code. Key concepts I learnt include using artifact repositories, connecting Maven with CodeArtifact, setting up IAM permissions to give the EC2 Instances the permissions to access CodeArtifact.

Project reflection

This project took me approximately 2.5 hours including demo time and Troubleshooting time. The most challenging part was to understand new concepts that were brand new in my ears whic came with few errors with the CLI Commands around file names and permission settings. It was most rewarding to see CodeArtifact fill up with heaps of packages when the connection was all set up.

This project is part three of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project in the next few comming days.

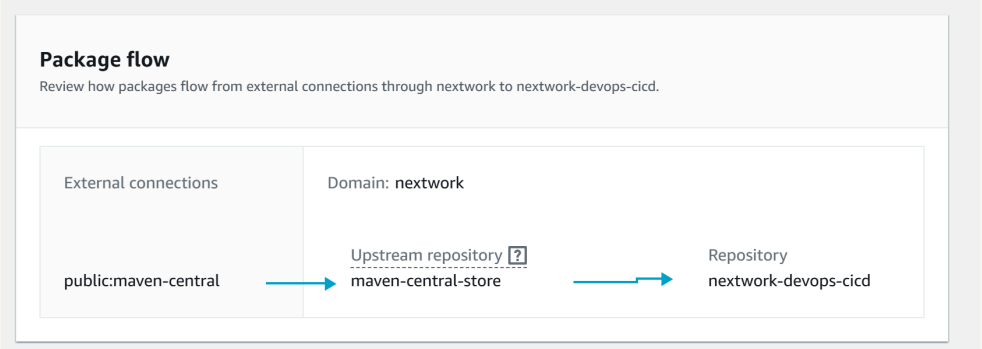
Kayira Bertrand
NextWork Student

CodeArtifact Repository

CodeArtifact is an Artifact Repository service, which means we can use it to create repositories to store our Web App's packages and dependencies. Engineering teams use artifact repositories for security, control and reliability.

A domain is like a folder that holds together multiple repositories under the same project or organisation. They are helpful for setting up permissions for multiple CodeArtifact repositories in one go. My domain is called nextwork.

A CodeArtifact repository can have an upstream repository, which means a public source of packages that Maven can visit if they can't find it in our local CodeArtifact repository. My repository's upstream repository is Maven Central Store which is the largest Java Repository and is extremely helpful when building a Java Web App.



Kayira Bertrand
NextWork Student

CodeArtifact Security

Issue

To access CodeArtifact, I need an Authentication token that allows my EC2 Instance to have the permissions to access CodeArtifact Repositories for the next 12 hours. I ran into an error when retrieving a token because my EC2 Instance does not have the permission to access my AWS Resources by default, this follows the principle of least priviledge.

Resolution

To resolve the error with my security token, I set up an IAM Policy that grants access to CodeArtifact and then an IAM role that I can attach to my EC2 Instance. This resolved the error because my EC2 Instance now has access to request an authorization from the repository.

It's security best practice to use IAM roles because IAM Roles are more secure and scalable compared to hardcoded credentials. Hardcoded Credentials are much more vulnerable to Security attacks and getting misused resulting errors, losses and delays. IAM roles are applied via AWS and do not require any hardcoding of credintials which much more secure and encrypted. IAM Roles are also the better option if you want to give the same set of roles to many different Instances.

Kayira Bertrand
NextWork Student

The JSON policy attached to my role

The JSON policy I set up grants to CodeArtifact. Specifically it grants access to three key actions related to CodeArtifact - Retrieving an authorization token, finding the repository (endpoint) and viewing the packages inside the repository.

Policy editor Visual

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "codeartifact:GetAuthorizationToken",
                "codeartifact:GetRepositoryEndpoint",
                "codeartifact:ReadFromRepository"
            ],
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": "sts:GetServiceBearerToken",
            "Resource": "*",
            "Condition": {
                "StringEquals": {
                    "sts:AWSServiceName": "codeartifact.amazonaws.com"
                }
            }
```

Kayira Bertrand
NextWork Student

Maven and CodeArtifact

To test the connection between Maven and CodeArtifact, I compiled my web app using settings.xml

The settings.xml file configures Maven to use my CodeArtifact Repository. It applies Maven with the name and Authentication token to get access to the CodeArtifact repository. It also sets up a profile section in case I ever have multiple repositories and I need to tell Maven when to use which one.

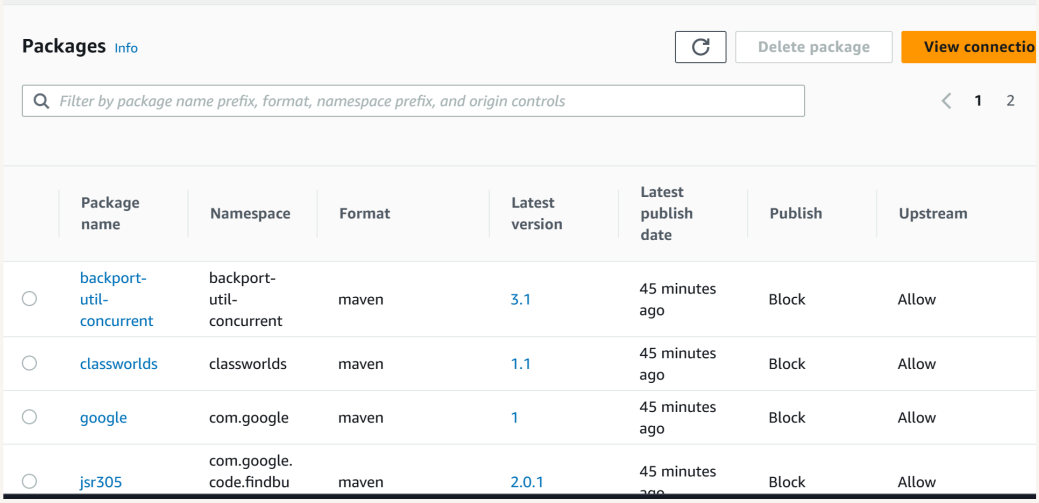
Compiling means the process of translating my Web App source code into something that machine can run. Maven is my compiler, so I asked maven to compile my Web App code! While it's compiling, Maven will need to put together all the packages that my Web App needs to run. To retrieve these packages, Maven visits CodeArtifacts (who will say that it doesn't have any packages available then Maven would go to request the exact same packages from the upstream repository [Maven Central] and once Maven has those packages store local copies the CodeArtifact Repository).

settings.xml

```
<settings>
    <servers>
    <server>
        <id>nextwork-nextwork-devops-cicd</id>
        <username>aws</username>
        <password>${env.CODEARTIFACT_AUTH_TOKEN}</password>
    </server>
    </servers>
    <profiles>
    <profile>
        <id>nextwork-nextwork-devops-cicd</id>
        <activation>
        <activeByDefault>true</activeByDefault>
        </activation>
        <repositories>
        <repository>
            <id>nextwork-nextwork-devops-cicd</id>
            <url>https://nextwork-851725390806.d.codeartifact.us-west-2.amazonaws.com/maven/nextwork-devops-cicd/</url>
        </repository>
        </repositories>
    </profile>
    </profiles>
    <mirrors>
    <mirror>
        <id>nextwork-nextwork-devops-cicd</id>
```

Verify Connection

After compiling, I checked my CodeArtifact Repository and I noticed 4 pages of Packages Inside. This tells me that I successfully store my Web App dependencies in a CodeArtifact repository.



Uploading My Own Packages

In a project extension, I also decided to become a package publisher - which means publishing my own packages into my CodeArtifact Repository. This is useful in situations where you might have internal team members developing their own packages and wanting their fellow teammates to have access to it (without giving the entire world access).

To create my own package, I set up a text file and used tar package up that text file. I also generated a security hash because that will tell give CodeArtifact a way to figure out whether or not the package has been tampered with in transit. If it has been tampered with, the security hash that CodeArtifact generates does not matchup to the security hash I generate.

To publish the package, I ran a CLI command that let's me upload the package I created in CloudShell to my repository. When I look at the package details in CodeArtifact, I can see the version number, publish date and even the origin (In this case the CodeArtifact repository itself is the origin).

To validate my packages, I then tried to download the package back into the CloudShell Terminal, I saw a package successfully downloaded/Installed from CodeArtifact and then I unzipped the package and read what was inside. I could see the text that I wrote inside the text file before I packaged it up. This proves to me that CodeArtifact successfully stored one of my own custom packages that I can retrieve later on as well.

```
{
    "format": "generic",
    "namespace": "secret-mission",
    "package": "secret-mission",
    "version": "1.0.0",
    "versionRevision": "0mahSUnBMnewFAOpZ7DzN4qFKmxi0gRU8eHGmCChYks=",
    "status": "Published",
    "asset": {
        "name": "secret-mission.tar.gz",
        "size": 168,
        "hashes": {
            "MD5": "e5e4994238dafb3789fffd629a9f013b",
            "SHA-1": "780c6dcc833841a522586bdce6636f9853a19010",
            "SHA-256": "c581bbf870f6caffc1f8db5ee351016bf228bdd28663b058b6092d81fe454420",
            "SHA-512": "3eb4af25d7533f94ed87bae18ed5c77f638f46b108a7d421026b859922aa21c3e4e38346553209a19a01cb43dfc54e12c5cafd49040b2f450821afb157adfc60"
        }
    }
}
```

The place to learn & showcase your skills

Check out nextwork.org for more projects